győri szakképzési centrum

Jedlik Ányos
Gépipari és Informatikai
Technikum és Kollégium

9021 Győr, Szent István út 7.
+36 (96) 529-480
+36 (96) 529-448

OM: 203037/003
jedlik@jedlik.eu
www.jedlik.eu

Záródolgozat feladatkiírás

Tanuló(k) neve:	Roncz Olivér, Szemethy Levente
Képzés:	nappali
Szak:	5 0613 12 03 Szoftverfejlesztő és -tesztelő technikus

A záródolgozat címe:

Romándi Vadászház Klinik

Konzulens:	Soós Gábor
Beadási határidő:	2026. 04. 20.

Győr, 2026. április 20.

Módos Gábor
igazgató

győri szakképzési centrum

Jedlik Ányos
Gépipari és Informatikai
Technikum és Kollégium

9021 Győr, Szent István út 7.
+36 (96) 529-480
+36 (96) 529-448

OM: 203037/003
jedlik@jedlik.eu
www.jedlik.eu

Konzultációs lap

	A konzultáció		Konzulens aláírása
	ideje	témája	
1.	2026.02.15.	Témaválasztás és specifikáció	
2.	2026.03.14.	Záródolgozat készültségi fokának értékelése	
3.	2026.04.20.	Dokumentáció véglegesítése	

Tulajdonosi nyilatkozat

Ez a dolgozat a saját munkánk eredménye. Dolgozatunk azon részeit, melyeket más szerzők munkájából vettünk át, egyértelműen megjelöltük.
Ha kiderülne, hogy ez a nyilatkozat valótlan, tudomásul vesszük, hogy a szakmai vizsgabizottság a szakmai vizsgáról kizár minket és szakmai vizsgát csak új záródolgozat készítése után tehetünk.

Győr, 2026. április 20.

_____
tanuló aláírása

_____
tanuló aláírása

Szerepkör	Leírás	Fő Engedélyek
DOCTOR	Orvos/Szakorvos	Bejelentkezés, profil kezelés, rendelési idő szerkesztése, szolgáltatások kezelése, páciensek kezelése, leletek rögzítése
ADMIN	Rendszeradminisztrátor	Teljes hozzáférés minden funkcióhoz

---

Rendszerarchitektúra

2.1 Full-Stack Architektúra Diagramm

```
┌─────────────────────────────────────────────────────────────────────┐
│                    FRONTEND LAYER (React/Next.js)                   │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ Pages:                                                       │   │
│  │ - Login / Registration                                       │   │
│  │ - Dashboard (Patient/Doctor/Admin)                           │   │
│  │ - Appointments Management                                    │   │
│  │ - Medical Records & Leletek                                  │   │
│  │ - Doctor Services & Booking                                  │   │
│  │ - Timetable Management                                       │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                     │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ State Management: Zustand (globalStore)                      │   │
│  │ - Auth State (User, Token)                                   │   │
│  │ - Persistence (localStorage)                                 │   │
│  └──────────────────────────────────────────────────────────────┘   │
│                                                                     │
│  ┌──────────────────────────────────────────────────────────────┐   │
│  │ UI Components: DaisyUI + Tailwind CSS                        │   │
│  │ - Responsive Design                                          │   │
```

```
│   │  - Form Inputs & Validation                          │   │
│   │  - Date/Time Pickers                                  │   │
│   └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                          ↕ (HTTP/JSON)
                     [JWT Token Auth]
┌──────────────────────────────────────────────────────────────┐
│                   BACKEND API LAYER (Express)                 │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  REST API Endpoints:                                  │   │
│   │  - /api/users (Auth, Profile)                         │   │
│   │  - /api/appointments (Booking, Management)            │   │
│   │  - /api/records (Medical Records)                     │   │
│   │  - /api/services (Doctor Services)                    │   │
│   │  - /api/availability (Doctor Schedule)                │   │
│   │  - /api/admin (Admin Functions)                       │   │
│   └──────────────────────────────────────────────────────┘   │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  Middleware:                                          │   │
│   │  - Authentication (JWT)                               │   │
│   │  - Role-Based Access Control (RBAC)                   │   │
│   │  - Validation & Error Handling                        │   │
│   └──────────────────────────────────────────────────────┘   │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  Services:                                            │   │
│   │  - Email Service (Nodemailer)                         │   │
│   │  - PDF Generation                                     │   │
│   └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
                          ↕ (Mongoose ODM)
┌──────────────────────────────────────────────────────────────┐
│              DATABASE LAYER (MongoDB Atlas)                   │
│   ┌──────────────────────────────────────────────────────┐   │
│   │  Collections:                                         │   │
│   │  - users (Patients, Doctors, Admins)                  │   │
│   │  - appointments (Booking Records)                     │   │
│   │  - records (Medical Records)                          │   │
│   │  - services (Doctor Services)                         │   │
│   │  - availabilities (Doctor Schedule)                   │   │
│   └──────────────────────────────────────────────────────┘   │
└──────────────────────────────────────────────────────────────┘
```

2.2 Kommunikációs Flow

Bejelentkezési Flow:

```
1. Frontend → POST /api/users/login (email, password)
2. Backend → Valida jelszót, generál JWT tokent
3. Backend ← Response: { user, token }
4. Frontend → Token localStorage-ben tárolás, Zustand state update
5. Frontend → Átirányítás /dashboard, összes kéréshez Authorization header
```

Időpont Foglalási Flow:

```
1. Frontend → GET /api/availability/doctor/{doctorId} (rendelési idők)
2. Frontend → GET /api/services/{doctorId} (szolgáltatások)
3. User → Dátum + Idő kiválasztása
4. Frontend → POST /api/appointments (booking request)
5. Backend → Valida rendelkezésre állást, létrehozza az appointmentot
6. Backend → Email küldés páciensnek és orvosnak
7. Frontend ← Response: { appointment }
8. Frontend → Átirányítás /appointments, lista frissítés
```

Technológiai Stack (Full-Stack)

3.1 Frontend Stack

Kategória	Technológia	Verzió	Célja
Framework	Next.js	16.1.6	React framework SSR/SSG-vel
UI Library	React	19.2.3	UI komponensek és state
State Management	Zustand	v5	Global state, localStorage persist
HTTP Client	Axios	v1	API kommunikáció
CSS Framework	Tailwind CSS	v4	Utility-first CSS
Component Library	DaisyUI	v5	Előre built komponensek
Date/Time	dayjs	v1	Dátum formázás
Date Picker	react-datepicker	v9.1.0	Dátum kiválasztás
Time Picker	react-time-picker	v8.0.2	Idő kiválasztás
Notifications	react-hot-toast	v2.6.0	Toast üzenetek
Icons	lucide-react	v0.563.0	SVG ikonok
Dev Tools	TypeScript, ESLint, Prettier	Latest	Code quality
Testing	Cypress	v15.13.0	E2E tesztelés

3.2 Backend Stack

Kategória	Technológia	Verzió	Célja
Runtime	Node.js	v22	JavaScript futási környezet
Framework	Express.js	v4.x	RESTful API kiszolgálása
Database	MongoDB Atlas	Cloud	NoSQL adatbázis
ODM	Mongoose	v7+	Database modeling
Authentication	JSON Web Token (JWT)	v9	Stateless autentikáció
Password Hashing	bcryptjs	Latest	Jelszó titkosítás
Email Service	Nodemailer	Latest	Email küldés
Environment	dotenv	Latest	Env. változók
Validation	Mongoose Validators	Built-in	Schema validáció
API Documentation	Swagger/OpenAPI	Latest	API docs
Testing	Jest	Latest	Unit/Integration tesztek
Hosting	Vercel	Cloud	Backend deployment

⍰ II. BACKEND DOKUMENTÁCIÓ

Backend Projekt Szerkezete

4.1 Backend Könyvtárstruktúra

```
backend/
├── server.js                          # Fő alkalmazás fájl
├── package.json                       # NPM függőségek
├── .env                               # Környezeti változók (nicht versionál
va)
├── .env.example                       # Environment template
├── .gitignore
│
├── config/
│   └── db.js                          # MongoDB kapcsolat konfigurálása
│
├── models/                            # Mongoose Schemák
```

```
│   ├── User.js                        # Felhasználó séma
│   ├── Appointment.js                 # Időpont séma
│   ├── Record.js                      # Orvosi rekord séma
│   ├── Service.js                     # Szolgáltatás séma
│   └── Availability.js                # Rendelkezésre állás séma
│
├── routes/                            # API Endpoint definíciók
│   ├── userRoutes.js                  # Felhasználó végpontok
│   ├── appointmentRoutes.js           # Időpont végpontok
│   ├── recordRoutes.js                # Rekord végpontok
│   ├── serviceRoutes.js               # Szolgáltatás végpontok
│   └── availabilityRoutes.js          # Rendelkezésre állás végpontok
│
├── middleware/                        # Middleware komponensek
│   ├── authMiddleware.js              # JWT hitelesítés, RBAC
│   └── errorMiddleware.js             # Centralizált hibakezelés
│
├── mail/                              # Email funkciók
│   └── mail.js                        # Email sablonok és küldés
│
├── docs/                              # Swagger/OpenAPI dokumentáció
│   ├── user.swagger.yaml
│   ├── appointment.swagger.yaml
│   ├── record.swagger.yaml
│   ├── service.swagger.yaml
│   └── availability.swagger.yaml
│
└── jest/                              # Tesztek
    ├── setup.js
    └── user.test.js
```

4.2 Backend Fájlok Célja

Fájl	Célja
server.js	Express app inicializálása, adatbázis csatlakozás, route regisztráció, Swagger setup
config/db.js	MongoDB Atlas kapcsolat Mongoose-zal
models/*.js	Adatbázis entitások schemái és validáció
routes/*.js	API endpoint definíciók
middleware/authMiddleware.js	JWT validáció, role-based access control
middleware/errorMiddleware.js	Centralizált error handling, 400/401/403/500 kezelés
mail/mail.js	Email sablonjok és küldés Nodemailer-rel
docs/	Swagger/OpenAPI YAML fájlok
jest/	Unit és integration tesztek

Adatmodellek (Models)

5.1 User Model

Fájl: `models/User.js`

Mező	Típus	Kötelező	Leírás
_id	ObjectId	✅	MongoDB auto-generated
name	String	✅	Teljes név (3-50 kar.)
email	String	✅	Egyedi email (regex validált)
password	String	✅	Bcrypt titkosított (8+ kar., 1 szám, 1 speciális)
phone	String	✅	Telefonszám (+36...)
role	Enum	✅	ADMIN | DOCTOR | PATIENT
gender	String	❌	MALE | FEMALE
birthDate	Date	❌	Születési dátum (ISO format)
specialization	String	❌	Orvos szakterülete
tajNumber	String	❌	TAJ szám (9 számjegy, páciensek)
address	String	❌	Lakcím
records	[ObjectId]	❌	Record referenciák
availabilities	[ObjectId]	❌	Availability referenciák
resetPasswordCode	String	❌	Jelszó reset kód (6 jegy)
resetPasswordExpires	Date	❌	Reset kód lejárata

Validációs Szabályok: - Email: RFC 5322 regex, egyedi - Jelszó: Min. 8 kar., 1+ nagybet., 1+ szám, 1+ speciális karakter - Telefonszám: +36 + 8-9 szám - TAJ: 9 számjegy (páciensek) - Név: 3-50 karakter

5.2 Appointment Model

Mező	Típus	Leírás
doctor_id	ObjectId (User ref)	Orvosra mutató referencia
patient_id	ObjectId (User ref)	Páciensre mutató referencia
service_id	ObjectId (Service ref)	Szolgáltatásra mutató referencia

Mező	Típus	Leírás
startTime	Date	Időpont kezdete (ISO format, jövőbeli!)
endTime	Date	Időpont vége (> startTime!)
status	Enum	PENDING | ACCEPTED | REJECTED | PROPOSED | CANCELLED | COMPLETED
referral_type	String	SELF | DOCTOR
referred_by	ObjectId (User ref)	Melyik orvos irányította (ha DOCTOR ref)
createdAt, updatedAt	Date	Mongoose timestamps

Status Szabályok: - **PENDING** → Kezdeti állapot - **ACCEPTED** → Orvos/admin megerősítette - **REJECTED** → Orvos elutasította - **PROPOSED** → Orvos új időt javasol - **CANCELLED** → Lemondva - **COMPLETED** → Lezárult

5.3 Record Model

Mező	Típus	Leírás
patient	ObjectId (User ref)	Páciensre mutató referencia
doctor	ObjectId (User ref)	Orvosra mutató referencia
appointment_id	ObjectId	Kapcsolódó appointment (optional)
service	ObjectId (Service ref)	Szolgáltatásra mutató referencia
description	String	Lelet szövege, diagnosztikai megjegyzés
createdAt, updatedAt	Date	Mongoose timestamps

5.4 Service Model

Mező	Típus	Leírás
doctor_id	ObjectId (User ref)	Orvosra mutató referencia
topic	String	Szolgáltatás neve (pl. “Kardiológia”)
description	String	Részletes leírás
location	String	Vizsgáló helye (pl. “102-es vizsgáló”)
date	Date	Szolgáltatás dátuma
price	String	Ár (pl. “25000 Ft”)
patient_id	ObjectId	Specifikus pácienshez (optional)

5.5 Availability Model

Mező	Típus	Leírás
doctor	ObjectId (User ref)	Orvosra mutató referencia
dayOfWeek	Enum	Hétfő, Kedd, … Vasárnap
startTime	String	Kezdési idő (HH:mm format)

Mező	Típus	Leírás
endTime	String	Befejezési idő (HH:mm format)
slotDuration	Number	Idősáv hossza percben (min: 10)
isActive	Boolean	Aktív-e (default: true)

Unique Index: doctor + dayOfWeek (naponta egy periódus)

Middleware Rendszer

6.1 Auth Middleware

Fájl: `middleware/authMiddleware.js`

1. protect - JWT Hitelesítés

```
export const protect = async (req, res, next) => {
    let token;

    if (req.headers.authorization?.startsWith('Bearer')) {
        token = req.headers.authorization.split(' ')[1];

        try {
            const decoded = jwt.verify(token, process.env.JWT_SECRET);
            req.user = await User.findById(decoded.id).select('-password');
            next();
        } catch (error) {
            res.status(401).json({ message: 'Érvénytelen token' });
        }
    } else {
        res.status(401).json({ message: 'Nincs token' });
    }
};
```

2. admin - Admin Csak

```
export const admin = (req, res, next) => {
    if (req.user && req.user.role === 'ADMIN') {
        next();
    } else {
        res.status(403).json({ message: 'Admin jogosultság szükséges' });
    }
};
```

3. doctorOrAdmin - Orvos vagy Admin

```
export const doctorOrAdmin = (req, res, next) => {
    if (req.user && (req.user.role === 'DOCTOR' || req.user.role === 'ADMIN')
) {
        next();
    } else {
```

```
        res.status(403).json({ message: 'Orvos vagy admin jogosultság szükség
es' });
    }
};
```

6.2 Error Middleware

Fájl: `middleware/errorMiddleware.js`

```
export const errorHandler = (err, req, res, next) => {
    // Mongoose validációs hiba
    if (err.name === 'ValidationError') {
        const errors = Object.keys(err.errors).reduce((acc, key) => {
            acc[key] = err.errors[key].message;
            return acc;
        }, {});
        return res.status(400).json({ success: false, message: 'Validációs hi
ba', errors });
    }

    // Duplikált kulcs (11000 error code)
    if (err.code === 11000) {
        const field = Object.keys(err.keyValue)[0];
        return res.status(400).json({
            success: false,
            message: `${field} már foglalt`,
            errors: { [field]: `Ez az ${field} már használatban van` }
        });
    }

    // Szokásos szerver hiba
    res.status(err.statusCode || 500).json({
        success: false,
        message: err.message || 'Szerver hiba'
    });
};
```

Backend API Végpontok

7.1 Felhasználó Végpontok

POST /api/users/register - Regisztráció

Jogosultság: Nyilvános
Request:

```
{
  "name": "Kovács Béla",
  "email": "kovacs@pelda.hu",
```

```
  "password": "SecurePass123!",
  "phone": "+36301234567",
  "birthDate": "1990-01-01",
  "role": "PATIENT",
  "gender": "MALE",
  "tajNumber": "123456789",
  "address": "1051 Budapest, Erzsébet tér 1."
}
```

Response (201):

```
{
  "success": true,
  "_id": "507f1f77bcf86cd799439012",
  "name": "Kovács Béla",
  "email": "kovacs@pelda.hu",
  "role": "PATIENT",
  "token": "eyJhbGciOiJIUzI1NiJ9..."
}
```

POST /api/users/login - Bejelentkezés

Jogosultság: Nyilvános
Request:

```
{
  "email": "kovacs@pelda.hu",
  "password": "SecurePass123!"
}
```

Response (200): Felhasználó + JWT token

GET /api/users/doctors - Orvosok Listázása

Jogosultság: Nyilvános
Response (200):

```
[
  {
    "_id": "507f1f77bcf86cd799439014",
    "name": "Dr. Romándi Antal",
    "specialization": "Kardiológia",
    "email": "romandi@klinik.hu",
    "phone": "+36301112233"
  }
]
```

GET /api/users/stats/me - Saját Statistikák

Jogosultság: Hitelesített
Response (200):

```
{
  "stats": {
    "type": "PATIENT",
    "totalCompleted": 5,
    "totalActive": 2,
    "healthInvestment": 75000,
    "nextAppointment": { ... },
    "lastVisit": { ... }
  }
}
```

PUT /api/users/profile - Profil Frissítése

Jogosultság: Hitelesített
Request:

```
{
  "phone": "+36309876543",
  "address": "Budapest, Új utca 5.",
  "birthDate": "1990-03-20",
  "password": "NewPass123!" // optional
}
```

Response (200): Frissített user objektum

POST /api/users/forgot-password - Jelszó Reset Kérelem

Jogosultság: Nyilvános
Request:

```
{ "email": "kovacs@pelda.hu" }
```

Response (200): Email küldve (6 jegyű kóddal, 10 perc érvényes)

POST /api/users/reset-password - Jelszó Módosítása

Jogosultság: Nyilvános
Request:

```
{
  "email": "kovacs@pelda.hu",
  "code": "123456",
  "newPassword": "NewSecurePass123!"
}
```

Response (200): Jelszó módosítva

7.2 Időpont Végpontok

POST /api/appointments - Új Időpont Foglalása

Jogosultság: Hitelesített
Request:

```
{
  "doctor_id": "507f1f77bcf86cd799439014",
  "patient_id": "507f1f77bcf86cd799439012",
  "service_id": "507f1f77bcf86cd799439016",
  "startTime": "2026-03-15T09:00:00.000Z",
  "endTime": "2026-03-15T09:30:00.000Z"
}
```

Response (201): Teljes appointment objektum

Mellékhatás: Email a páciensnek és orvosnak

GET /api/appointments/my - Saját Időpontok

Jogosultság: Hitelesített
Response (200): Bejelentkezett felhasználó saját időpontjai

PUT /api/appointments/{id} - Időpont Módosítása

Jogosultság: Owner vagy admin
Request:

```
{
  "status": "ACCEPTED",
  "startTime": "2026-03-16T10:00:00.000Z"
}
```

Response (200): Módosított appointment

7.3 Lelet Végpontok

POST /api/records - Új Lelet Rögzítése

Jogosultság: DOCTOR vagy ADMIN
Request:

```
{
  "patient": "507f1f77bcf86cd799439012",
  "appointment_id": "507f1f77bcf86cd799439015",
  "service": "507f1f77bcf86cd799439016",
  "description": "Szívultrahang: normális. Kontroll 6 hónap múlva."
}
```

Response (201): Teljes record objektum

GET /api/records/my - Saját Leletek

Jogosultság: Hitelesített
Response (200): - Ha PATIENT: Saját leletek (orvosok által feljegyzett) - Ha DOCTOR: Az ő által feljegyzett leletek

GET /api/records/{id}/pdf - Lelet PDF Letöltése

Jogosultság: Owner vagy admin
Response (200): PDF binary

7.4 Szolgáltatás Végpontok

GET /api/services/{doctorId} - Orvos Szolgáltatásai

Jogosultság: Nyilvános
Response (200):

```
[
  {
    "_id": "507f1f77bcf86cd799439016",
    "doctor_id": "507f1f77bcf86cd799439014",
    "topic": "Szív ultrahang",
    "description": "30 perc konzultáció + EKG",
    "location": "102-es vizsgáló",
    "price": "25000"
  }
]
```

POST /api/services - Új Szolgáltatás

Jogosultság: DOCTOR vagy ADMIN
Request:

```
{
  "doctor_id": "507f1f77bcf86cd799439014",
  "topic": "Szív ultrahang",
  "description": "30 perc konzultáció + EKG",
  "location": "102-es vizsgáló",
  "date": "2026-05-20T10:00:00.000Z",
  "price": "25000"
}
```

Response (201): Teljes service objektum

PUT /api/services/{id} - Szolgáltatás Módosítása

Jogosultság: Owner (doctor) vagy admin
Response (200): Módosított service

DELETE /api/services/{id} - Szolgáltatás Törlése

Jogosultság: Owner vagy admin
Response (200): Sikeresen törölve

7.5 Rendelkezésre Állás Végpontok

POST /api/availability - Új Rendelkezésre Állás

Jogosultság: DOCTOR vagy ADMIN
Request:

```
{
  "doctor": "507f1f77bcf86cd799439014",
  "dayOfWeek": "Hétfő",
  "startTime": "08:30",
  "endTime": "12:00",
  "slotDuration": 30
}
```

Response (201): Teljes availability objektum

GET /api/availability/my - Saját Rendelkezésre Állás

Jogosultság: DOCTOR vagy ADMIN
Response (200): Orvos összes rendelkezésre állási periódusa

GET /api/availability/doctor/{doctorId} - Orvos Rendelkezésre Állása

Jogosultság: Nyilvános
Response (200):

```
{
  "data": [
    {
      "_id": "507f1f77bcf86cd799439018",
      "dayOfWeek": "Hétfő",
      "startTime": "08:30",
      "endTime": "12:00"
    }
  ]
}
```

DELETE /api/availability/{id} - Rendelkezésre Állás Törlése

Jogosultság: Owner vagy admin
Response (200): Sikeresen törölve

Email Rendszer

8.1 Email Konfiguráció

Provider: Gmail + Nodemailer
Feladó: romandi.klinik@gmail.com
Auth: App Password (Gmail biztonsági beállítások)

```
const getTransporter = () => {
    return nodemailer.createTransport({
        service: "gmail",
        auth: {
            user: "romandi.klinik@gmail.com",
            pass: process.env.EMAIL_PASS
        }
    });
};
```

8.2 Email Típusok

Email Típus	Meghívás	Tárgy	Tartalom
Welcome	Regisztráció után	Sikeres regisztráció	Üdvözlés, bejelentkezési link
Booking	Időpont foglalása után	Időpontfoglalás rögzítve	Foglalás részletei, referencia szám
Booking Confirmation	Orvos megerősítése után	Időpont megerősítve	Végső megerősítés
Doctor Response	Lelet rögzítése után	Orvosi válasz érkezett	Lelet szövege, vizsgálat eredménye
Password Reset	Jelszó reset kérelem	Jelszó reset kód	6 jegyű kód, 10 perc érvényes
Profile Modify	Profil frissítés után	Profil módosítása	Módosítás megerősítése
Account Delete	Fiók törlés után	Fiók törlése	Törlés megerősítése

---

▯ III. FRONTEND DOKUMENTÁCIÓ

Frontend Projekt Szerkezete

9.1 Frontend Könyvtárstruktúra

```
frontend/
├── app/                                # Next.js App Router
│   ├── layout.tsx                      # Root layout (HTML structure)
```

```
│   ├── page.tsx                        # Login oldal (/)
│   ├── globals.css                     # Globális stílusok
│   │
│   ├── dashboard/
│   │   └── page.tsx                    # Dashboard (user-specifikus)
│   │
│   ├── appointments/
│   │   └── page.tsx                    # Időpontok kezelése
│   │
│   ├── doctors/
│   │   └── page.tsx                    # Orvosok listája (PATIENT)
│   │
│   ├── doctorservices/
│   │   └── [doctorId]/
│   │       └── page.tsx                # Orvos szolgáltatásai + foglalás
│   │
│   ├── infos/
│   │   └── page.tsx                    # Orvosi leletek
│   │
│   ├── newinfo/
│   │   └── [appointmentId]/
│   │       └── page.tsx                # Új lelet rögzítése (DOCTOR)
│   │
│   ├── myservices/
│   │   └── page.tsx                    # Saját szolgáltatások (DOCTOR)
│   │
│   ├── timetable/
│   │   └── page.tsx                    # Rendelési idő kezelése (DOCTOR)
│   │
│   ├── registration/
│   │   └── page.tsx                    # Felhasználó regisztráció
│   │
│   ├── forgot-password/
│   │   └── page.tsx                    # Jelszó reset kérelem
│   │
│   ├── reset-password/
│   │   └── page.tsx                    # Jelszó reset
│   │
│   └── header/
│       └── page.tsx                    # Navigációs header
│
├── store/
│   └── globalStore.ts                  # Zustand global state management
│
├── types/
│   └── global.d.ts                     # TypeScript típusdefiníciók
│
├── public/                             # Statikus fájlok
│
├── cypress/                            # E2E tesztek
```

```
│   ├── e2e/                          # Teszt szekvenciák
│   ├── fixtures/                     # Teszt adatok
│   └── support/                      # Cypress támogatás
│
├── package.json                      # NPM függőségek
├── tsconfig.json                     # TypeScript konfigurálás
├── next.config.ts                    # Next.js konfigurálás
├── tailwind.config.js                # Tailwind CSS theme
├── postcss.config.mjs                # PostCSS plugin konfigurálás
├── eslint.config.mjs                 # ESLint szabályok
├── prettier.config.cjs               # Prettier formázási szabályok
├── cypress.config.ts                 # Cypress E2E konfigurálás
└── README.md                         # Projekt README
```

Globális Állapotkezelés (Zustand)

10.1 Store Szerkezete

Fájl: `store/globalStore.ts`

```typescript
type User = {
  id: string;                                  // MongoDB _id
  name: string;                                // Teljes név
  email: string;                               // Email cím
  role: "ADMIN" | "DOCTOR" | "PATIENT";       // Felhasználó típus
  phone: string;                               // Telefonszám
  address: string;                             // Lakcím
  tajNumber: string;                           // TAJ szám
  birthDate?: string;                          // Születési dátum (optional)
  gender?: "MALE" | "FEMALE";                  // Nem (optional)
  specialization?: string;                     // Orvosi specialitás (optional)
};

type GlobalStore = {
  user: User | null;                           // Bejelentkezett felhasználó
  token: string | null;                        // JWT authentikációs token
  lightTheme: boolean;                         // Téma beállítás

  setAuth: (user: User, token: string) => void;  // Bejelentkezés
  logout: () => void;                            // Kijelentkezés
  setLightTheme: (value: boolean) => void;       // Téma váltás
};
```

10.2 localStorage Persistálás

- **Store kulcsa:** `auth-storage`
- **Automatikus helyreállítás:** Oldal újratöltéskor megmarad az auth state
- **Middleware:** `persist()` Zustand middleware

10.3 Store Használata

```
// Bejelentkezés után:
const { setAuth } = useGlobalStore();
setAuth({
  id: data._id,
  name: data.name,
  email: data.email,
  role: data.role,
  // ... további mezők
}, data.token);

// Kijelentkezés:
const { logout } = useGlobalStore();
logout();

// Adatok olvasása:
const { user, token } = useGlobalStore();
```

Frontend Oldalak és Komponensek

11.1 Bejelentkezési Oldal (/)

Komponens: `app/page.tsx`

Funkciók:

- Email/jelszó input validáció
- Enter billentyű kezelés (animációval)
- Jelszó maszkolás (□ / □ gomb)
- Regisztrációra és jelszó resetra navigálás

API Integrációk:

- `POST /api/users/login` - Bejelentkezés

UI Elemek:

- Email input
- Jelszó input (toggle visibility)
- "Belépés" gomb (animációval)
- "Regisztráció" gomb
- "Elfelejtetted a jelszavad?" link
- Hiba üzenet megjelenítés

11.2 Dashboard (/dashboard)

Komponens: `app/dashboard/page.tsx`

PATIENT Dashboard:

- Statistikák: befejezett, aktív időpontok, egészségügyi beruházás
- Következő időpont megjelenítése
- Profil szerkesztése lehetőség
- Jelszó módosítása

DOCTOR Dashboard:

- Statistikák: befejezett, aktív időpontok, összes páciens
- Következő páciens (időpont)
- Utolsó vizsgálat
- Profil szerkesztése

API Integrációk:

- `GET /api/users/stats/me` - Statistikák
- `PUT /api/users/profile` - Profil frissítés

11.3 Orvosok Listája (`/doctors`)

Komponens: `app/doctors/page.tsx`

Korlátozás:

- Csak PATIENT szerepkör érhet el
- DOCTOR/ADMIN átirányítás: `/dashboard`

Megjelenítés:

- Kártya alapú grid (1 col mobile, 2 tablet, 3 desktop)
- Orvos adatok: név, specialitás, tel., email
- "Időpontfoglalás" gomb

API Integrációk:

- `GET /api/users/doctors` - Orvosok listázása

Navigáció:

- Gombra kattintás: `/doctorservices/{doctorId}`

11.4 Orvos Szolgáltatásai és Foglalás (`/doctorservices/[doctorId]`)

Komponens: `app/doctorservices/[doctorId]/page.tsx`

Funkciók:

1. **Szolgáltatások megjelenítése**
   - Téma, leírás, helyszín, ár
   - "Foglalás" gomb
2. **Foglalás modális**

  - Dátum kiválasztása (react-datepicker)
  - Óra kiválasztása (08:00-17:00)
  - Perc kiválasztása (0, 10, 20, 30, 40, 50)
  - Validáció

API Integrációk:

- `GET /api/services/{doctorId}` - Szolgáltatások
- `GET /api/availability/doctor/{doctorId}` - Rendelési idők
- `POST /api/appointments` - Foglalás

Validáció:

```
if (!selectedDate || !selectedHour || !selectedMinute) {
  setError("Válassz dátumot és időpontot!");
}
```

11.5 Időpontok Kezelése (`/appointments`)

Komponens: `app/appointments/page.tsx`

Funkciók:

- Saját időpontok listázása
- Status szűrés (PENDING, ACCEPTED, REJECTED, PROPOSED, CANCELLED, COMPLETED)
- Paginálás (5 / oldal)
- Status módosítása

Status Tárgyak:

```
const statusLabels = {
  PENDING: "Függőben",
  ACCEPTED: "Elfogadva",
  REJECTED: "Elutasítva",
  PROPOSED: "Javasolt",
  CANCELLED: "Lemondva",
  COMPLETED: "Befejezve"
};
```

Akciógombok:

- **PATIENT:**
  - PENDING: “Lemondás”
  - PROPOSED: “Elfogadom” / “Elutasítom”
- **DOCTOR:**
  - PENDING: “Elfogadom” / “Elutasítom” / “Módosítás”

API Integrációk:

- `GET /api/appointments/my` - Saját időpontok

- `PUT /api/appointments/{id}` - Status módosítás

11.6 Orvosi Leletek (`/infos`)

Komponens: `app/infos/page.tsx`

Funkciók:

- Leletek listázása (paginálva: 6 / oldal)
- PATIENT: Saját leletek
- DOCTOR: Páciensek leleteik
- PDF letöltés (PATIENT)

Lelet Adatok:

- Páciens neve (DOCTOR view)
- Orvos neve, specialitás
- Dátum
- Lelet szövege (description)
- Szolgáltatás típusa

API Integrációk:

- `GET /api/records/my` - Saját leletek
- `GET /api/records/{id}/pdf` - PDF letöltés

11.7 Új Lelet Rögzítése (`/newinfo/[appointmentId]`)

Komponens: `app/newinfo/[appointmentId]/page.tsx`

Korlátozás:

- Csak DOCTOR szerepkör

Funkciók:

- Appointment adatok megjelenítése (read-only)
- Lelet szöveg bevitele (textarea)
- Lelet mentése

API Integrációk:

- `POST /api/records` - Lelet rögzítése
- `PUT /api/appointments/{id}` - Status COMPLETED-re állítás

11.8 Saját Szolgáltatások (`/myservices`)

Komponens: `app/myservices/page.tsx`

Korlátozás:

- Csak DOCTOR szerepkör
- PATIENT/ADMIN: átirányítás `/dashboard`

Funkciók:

- Saját szolgáltatások listázása (paginálva: 4 / oldal)
- Új szolgáltatás hozzáadása
- Szolgáltatás szerkesztése
- Szolgáltatás törlése

Form Mezők:

- Topic (kötelező)
- Description (kötelező)
- Location (kötelező)
- Price (kötelező, szám)

API Integrációk:

- `GET /api/services/{doctorId}` - Saját szolgáltatások
- `POST /api/services` - Új szolgáltatás
- `PUT /api/services/{id}` - Szerkesztés
- `DELETE /api/services/{id}` - Törlés

11.9 Rendelési Idő Kezelése (`/timetable`)

Komponens: `app/timetable/page.tsx`

Korlátozás:

- Csak DOCTOR szerepkör

Funkciók:

- Nap kiválasztása (Hétfő-Vasárnap)
- Kezdési idő (08:00-16:00)
- Befejezési idő (08:00-16:00)
- Rendelkezésre állás hozzáadása
- Meglévő periódusok listázása

UI Elemek:

- Select nap kiválasztása
- Gombsor idő kiválasztásához
- Active gomb: sárga (`bg-yellow-400`)

API Integrációk:

- `POST /api/availability` - Rendelkezésre állás hozzáadása
- `GET /api/availability/my` - Saját rendelkezésre állás

- `DELETE /api/availability/{id}` - Törlés

11.10 Regisztráció (`/registration`)

Komponens: `app/registration/page.tsx`

Szerepkör Kiválasztása:

- "Páciens szerekezz meg" → PATIENT form
- "Orvosként szeretnék regisztrálni" → DOCTOR form

PATIENT Form:

- name, email, password, phone, birthDate, gender, tajNumber, address

DOCTOR Form:

- name, email, password, phone, birthDate, gender, specialization

Validáció:

```
if (!role) return;
if (!topic || !description || !location || !price) {
  setErrorMessage("Minden mező kitöltése kötelező!");
}
```

API Integrációk:

- `POST /api/users/register` - Regisztráció

11.11 Jelszó Helyreállítás (`/forgot-password`)

Komponens: `app/forgot-password/page.tsx`

Funkciók:

- Email megadása
- "Kód küldése" gomb

API Integrációk:

- `POST /api/users/forgot-password` - Kód küldés

Navigáció:

- Siker után: `/reset-password?email={email}`

11.12 Jelszó Visszaállítása (`/reset-password`)

Komponens: `app/reset-password/page.tsx`

Funkciók:

- Email (read-only, URL-ből)

- Reset kód (6 jegy)
- Új jelszó
- Jelszó megerősítés

Validáció:

```
if (password !== confirm) {
  setError("A jelszavak nem egyeznek!");
}
```

API Integrációk:

- `POST /api/users/reset-password` - Jelszó reset

11.13 Header / Navigáció

Komponens: `app/header/page.tsx`

UI Elemek:

- **Desktop Menü (lg+):**
  - "Személyes" (dashboard)
  - "Orvosok" (csak PATIENT)
  - "Időpontok"
  - "Orvosi jelentések"
  - "Rendelési idő" (csak DOCTOR)
  - "Szolgáltatásaim" (csak DOCTOR)
- **Mobil Menü:**
  - Hamburger gomb (☰)
  - Legördülő menü
- **Felhasználó Info:**
  - Felhasználó neve
  - Felhasználó típusa (ADMIN/ORVOS/PÁCIENS)
  - "Kijelentkezés" gomb

Aktív Route Kiemelés:

```
className={pathname === path ? "bg-[#BF944A]" : "bg-[#A2A369]"}
```

UI Komponensek és Design System

12.1 Szín Paletta

```
/* Erdei téma */
--bg-dark-green: #36483D        /* Háttér */
--bg-brown: #6B4A2D             /* Panel/Card */
--color-gold: #BF944A           /* Primér (gomb, kiemelt) */
```

```
--color-light-gold: #A2A369     /* Szekunder */
--color-muted: #A89D62          /* Muted szöveg */
```

CSS Osztályok:

```
// Háttér
className="bg-[#36483D]"

// Panel
className="bg-[#6B4A2D]"

// Primér gomb
className="bg-[#BF944A] text-[#36483D]"

// Szekunder gomb
className="bg-[#A2A369] text-[#36483D]"

// Szöveg
className="text-[#BF944A]"    // Primér
className="text-[#A89D62]"    // Muted
```

12.2 Tailwind CSS Breakpoints

```
sm: 640px
md: 768px
lg: 1024px
xl: 1280px
2xl: 1536px
```

Responsive Pattern:

```
<div className="grid gap-6 md:grid-cols-2 lg:grid-cols-3">
  {/* Mobile: 1 col, Tablet: 2, Desktop: 3 */}
</div>
```

12.3 DaisyUI Komponensek

```
// Button
<button className="btn bg-[#BF944A] text-[#36483D]">
  Kattints ide
</button>

// Input
<input
  className="input input-bordered border-[#BF944A] bg-[#36483D]"
  placeholder="Bevitel"
/>

// Divider
<div className="divider text-[#BF944A]">vagy</div>
```

12.4 Tipográfia

```
text-xs:  12px (labels)
text-sm:  14px (small text)
text-base: 16px (body)
text-lg:  18px (section title)
text-2xl: 24px (page title)
text-3xl: 30px (heading)
text-5xl: 48px (large heading)
text-6xl: 60px (hero)
```

Font Weights:

```
font-normal:  400
font-medium:  500
font-bold:    700
```

⍰ IV. INTEGRÁCIÓ ÉS BIZTONSÁG

Hitelesítési Rendszer

13.1 Bejelentkezési Flow

```
1. Frontend → POST /api/users/login
2. Backend → Valida jelszót, generál JWT
3. Backend ← Response: { user, token }
4. Frontend → Zustand setAuth(), localStorage update
5. Frontend → Próba redirect /dashboard
6. Frontend → Összes request: Authorization: Bearer {token}
```

13.2 Token Management

Frontend:

```
// Bejelentkezés után
const { setAuth } = useGlobalStore();
setAuth(userData, token);  // Zustand + localStorage

// API kérésekhez
const { token } = useGlobalStore();
headers: { Authorization: `Bearer ${token}` }

// Kijelentkezéskor
const { logout } = useGlobalStore();
logout();  // Zustand + localStorage törlés
```

Backend:

```
// JWT generálás
const token = jwt.sign({ id: user._id }, process.env.JWT_SECRET, {
```

```
  expiresIn: "30d"
});

// JWT validáció (protect middleware)
const decoded = jwt.verify(token, process.env.JWT_SECRET);
req.user = await User.findById(decoded.id);
```

13.3 Token Érvényessége

- **Lejárati idő:** 30 nap
- **Lejárt token:** 401 Unauthorized hiba
- **Frontend kezelés:** Felhasználó átirányítása / bejelentkezési oldalra

---

Felhasználói Típusok és Hozzáférés Kontrol

14.1 PATIENT (Páciens) Engedélyek

Elérhető Oldalak: - `/` - Bejelentkezés ✅- `/registration` - Regisztráció ✅- `/dashboard` - Saját profil ✅- `/doctors` - Orvosok listája ✅- `/doctorservices/[doctorId]` - Foglalás ✅- `/appointments` - Saját időpontok ✅- `/infos` - Saját leletek ✅- `/forgot-password`, `/reset-password` ✅

Tiltott Oldalak (→ /dashboard): - `/myservices` - Saját szolgáltatások - `/timetable` - Rendelési idő - `/newinfo/[appointmentId]` - Lelet rögzítés

14.2 DOCTOR (Orvos) Engedélyek

Elérhető Oldalak: - `/` - Bejelentkezés ✅- `/registration` - Regisztráció ✅- `/dashboard` - Saját profil ✅- `/appointments` - Saját időpontok ✅- `/infos` - Páciensek leleteik ✅- `/myservices` - Saját szolgáltatások ✅- `/timetable` - Rendelési idő ✅- `/newinfo/[appointmentId]` - Lelet rögzítés ✅

Tiltott Oldalak (→ /dashboard): - `/doctors` - Orvosok listája - `/doctorservices/[doctorId]` - Foglalás

14.3 Frontend Hozzáférés Kontrol

Pattern 1: useEffect Védelme

```
useEffect(() => {
  if (user && user.role !== "EXPECTED_ROLE") {
    router.push("/dashboard");
  }
}, [user, router]);
```

Pattern 2: Conditional Render

```
if (!user || user.role !== "PATIENT") {
  return <div>Nincs jogosultság</div>;
}
```

Pattern 3: Dinamikus Menü

```
{user.role === "DOCTOR" && desktopBtn("/timetable", "Rendelési idő")}
```

Hibakezelés és Validáció

15.1 Frontend Validáció

Email Validáció:

```
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
if (!emailRegex.test(email)) {
  setError("Érvénytelen email cím");
}
```

Telefonszám Validáció:

```
// Minta: 06-tal kezd, 11 karakter
if (!/^06\d{9}$/.test(phone)) {
  setFieldErrors({ phone: "06 + 9 számjegy" });
}
```

Jelszó Validáció:

```
// Min. 8 kar., 1 nagybet., 1 szám, 1 speciális
if (password.length < 8 || !/[A-Z]/.test(password)) {
  setFieldErrors({ password: "Nem megfelelő jelszó" });
}
```

15.2 Backend Validáció

Mongoose Schema validáció + custom validators

```
const userSchema = new Schema({
  email: {
    type: String,
    required: "Email kötelező",
    unique: true,
    match: /regex/
  },
  password: {
    type: String,
    required: "Jelszó kötelező",
    validate: {
      validator: (v) => /regex/.test(v),
      message: "Jelszó nem felel meg"
    }
  }
});
```

15.3 Error Handling Pattern

Frontend:

```
try {
  setLoading(true);
  setError(null);

  const res = await fetch(url, options);
  const data = await res.json();

  if (!res.ok) {
    if (data.errors) {
      setFieldErrors(data.errors);  // Field-specifikus
    } else {
      throw new Error(data.message);  // Általános
    }
    return;
  }

  toast.success("Sikeresen létrehozva");
} catch (err) {
  const message = err instanceof Error ? err.message : "Hiba";
  setError(message);
  toast.error(message);
} finally {
  setLoading(false);
}
```

Backend:

```
if (!res.ok) {
  return res.status(err.statusCode || 500).json({
    success: false,
    message: err.message,
    errors: err.errors
  });
}
```

15.4 Toast Notifikációk

```
// Pozicionálás
<Toaster position="bottom-right" toastOptions={{ duration: 5000 }} />

// Típusok
toast.success("Sikeres!");
toast.error("Hiba!");
toast.loading("Betöltés...");
```

⍰ V. DEPLOYMENT ÉS TESZTELÉS

Telepítés és Üzembe Helyezés

16.1 Frontend Setup (Lokális)

```
# 1. Repository klónozása
git clone https://github.com/Szemethy/romandi-vadaszhaz-klinik-frontend.git
cd romandi-vadaszhaz-klinik-frontend

# 2. Függőségek telepítése
npm install

# 3. Fejlesztési szerver
npm run dev
# Elérhetőség: http://localhost:8080

# 4. Build
npm run build

# 5. Production indítása
npm start
```

16.2 Backend Setup (Lokális)

```
# 1. Repository klónozása
git clone https://github.com/Szemethy/romandi-vadaszhaz-klinik-backend.git
cd backend

# 2. Függőségek telepítése
npm install

# 3. .env fájl létrehozása
cp .env.example .env
# Szerkesztés: MONGO_URI, JWT_SECRET, EMAIL_PASS, stb.

# 4. Server indítása
npm start
# Elérhetőség: http://localhost:3000

# 5. Tesztelés
npm test
```

16.3 Environment Változók

Frontend (.env.local):

```
NEXT_PUBLIC_API_BASE_URL=https://romandi-vadaszhaz-klinik-backend.vercel.app
```

Backend (.env):

```
MONGO_URI=mongodb+srv://user:pass@cluster.mongodb.net/klinik
JWT_SECRET=your-super-secret-jwt-key-minimum-20-chars
EMAIL_PASS=your-gmail-app-password
FRONTEND_URL=https://frontend-app.vercel.app
NODE_ENV=production
```

16.4 Vercel Deployment

Frontend: 1. GitHub repo összekapcsolása 2. Vercel Dashboard: New Project → Import Git Repo 3. Framework: Next.js (auto-detect) 4. Build: `npm run build` 5. Deploy

Backend: 1. GitHub repo összekapcsolása 2. Vercel Dashboard: New Project 3. Build: `npm start` 4. Environment Variables beállítása

Production URLs: - Frontend: https://romandi-vadaszhaz-klinik-frontend-4.vercel.app/ - Backend: https://romandi-vadaszhaz-klinik-backend.vercel.app/

16.5 Code Quality

```
# ESLint
npm run lint

# Prettier format check
npm run format

# Prettier auto-fix
npm run format:fix
```

Tesztelés (E2E és Unit)

17.1 Cypress E2E Tesztelés

Frontend Tesztek:

```
cypress/e2e/
├── spec.cy.ts                  # Általános/demo
├── registration.cy.ts          # Regisztráció
├── dashboard.cy.ts             # Dashboard
├── doctors.cy.ts               # Orvosok
├── appointments.cy.ts          # Időpontok
├── infos.cy.ts                 # Leletek
└── ...
```

Futtatás:

```
# UI módban
npx cypress open

# Headless módban
npx cypress run
```

```
# Specifikus teszt
npx cypress run --spec "cypress/e2e/registration.cy.ts"
```

17.2 Backend Jest Tesztek

```
# Összes teszt
npm test

# Coverage
npm test -- --coverage

# Watch mód
npm test -- --watch
```

Fejlesztői Útmutató

18.1 Git Workflow

Branch Elnevezés:

```
feature/feature-name     - Új funkció
bugfix/bug-name         - Hibajavítás
hotfix/issue-name       - Azonnali javítás
```

Commit Üzenetek (Magyar):

```
Új: [feature] Bejelentkezési oldal elkészítése
Javítás: [bugfix] CSS layoutárak
Frissítés: [refactor] API hívások optimalizálása
```

18.2 Code Style

TypeScript:

```
// Explicit típusok
type User = {
  id: string;
  name: string;
  role: "ADMIN" | "DOCTOR" | "PATIENT";
};

// Function components
export default function ComponentName() {
  return (
    <div>
      {/* JSX */}
    </div>
  );
}
```

Tailwind Classes:

```
// Utility-first
className="flex gap-4 bg-[#36483D] text-white hover:bg-[#BF944A]"

// Mobile-first responsive
className="block md:grid lg:flex"
```

18.3 Új Oldal Hozzáadása

1. Könyvtár létrehozása:

```
app/newpage/
  └── page.tsx
```

2. Komponens sablon:

```
"use client";

import { useRouter } from "next/navigation";
import { useEffect } from "react";
import Header from "@/app/header/page";
import { useGlobalStore } from "@/store/globalStore";

export default function NewPagePage() {
  const router = useRouter();
  const { user } = useGlobalStore();

  // Védelme
  useEffect(() => {
    if (user && user.role !== "EXPECTED_ROLE") {
      router.push("/dashboard");
    }
  }, [user, router]);

  if (!user) return null;

  return (
    <div className="min-h-screen bg-[#36483D] text-[#A89D62]">
      <Header />
      <main className="mx-auto max-w-6xl p-8">
        <h1 className="text-3xl font-bold text-[#BF944A]">Oldal Cím</h1>
        {/* Tartalom */}
      </main>
    </div>
  );
}
```

3. Header-ben navigáció:

```
{user.role === "EXPECTED_ROLE" && desktopBtn("/newpage", "Oldal")}
```

18.4 Common Issues és Megoldások

Token Undefined:

```
if (!token) {
  router.push("/");
  return;
}
```

useRouter Hiba:

```
"use client";  // Server component → client component
import { useRouter } from "next/navigation";
```

CORS Hiba: - Backend CORS beállítása szükséges - Frontend: Vercel deployment előfeltétele

---

Összefoglalás

▯ Teljes Stack Funkciók

Modul	Backend	Frontend	Integráció
Autentikáció	JWT + bcrypt	Zustand store	Token header
Felhasználók	Models + routes	Pages + forms	CRUD операций
Időpontok	Appointments model	Appointments page	Booking flow
Leletek	Records model	Infos page	CRUD + PDF
Szolgáltatások	Services model	MyServices page	CRUD
Rendelkezésre Állás	Availability model	Timetable page	Schedule mgmt
Email	Nodemailer	Toast notifications	Automated emails

▯ Legfontosabb Pontok

✅**Moduláris Szerkezet:** Jól szervezett komponensek, routes, middleware
✅**Full-Stack Biztonság:** JWT, bcrypt, RBAC, validáció
✅**Responsive Design:** Mobile-first Tailwind CSS
✅**Type Safety:** TypeScript szinte mindenhol
✅**Error Handling:** Centralizált hibakezelés mind BE és FE
✅**Email Integráció:** Automatizált kommunikáció
✅**E2E Tesztelés:** Cypress tesztek beépítve
✅**Production Ready:** Vercel deployment, Docker ready

▯ Technológiai Összehasonlítás

Frontend: - React 19 + Next.js 16 (modern, production-grade) - Zustand (lightweight state management) - Tailwind CSS (utility-first, scalable) - DaisyUI (pre-built components)

Backend: - Express.js (proven, widespread) - MongoDB + Mongoose (flexible schema, scalable) - JWT (stateless authentication) - Nodemailer (reliable email service)

⍰ Performance & Scaling

- **Frontend:** CDN-on Vercel, SSR/SSG Next.js
- **Backend:** Stateless Express, load balancer ready
- **Database:** MongoDB Atlas cloud, auto-scaling
- **Email:** Async job queue (Nodemailer)

⍰ Dokumentáció Verzió

- **Verzió:** 1.0.0
- **Utolsó frissítés:** 2026. április
- **Backend:** v1.3.0+
- **Frontend:** v1.0.0+

⍰ Support és Kontakt

GitHub Repository: - Frontend: https://github.com/Szemethy/romandi-vadaszhaz-klinik-frontend - Backend: https://github.com/Szemethy/romandi-vadaszhaz-klinik-backend

API Dokumentáció: - Swagger/OpenAPI: https://romandi-vadaszhaz-klinik-backend.vercel.app/api-docs

Issues & Feature Requests: - GitHub Issues: https://github.com/Szemethy/

© 2026 Romándi Vadászház Klinik. Minden jog fenntartva.

Készítette: Roncz Olivér, Szemethy Levente
Licenc: Proprietary (Nem nyilvános)
Status: Production Ready ✅